Write a program to implement a semantic analyzer that performs type checking , ans semantic error detection on an abstract syntax tree(AST) produced by parser.

CODE:

```
#include <iostream>
#include <string>
#include <map>
#include <vector>
#include <sstream>
#include <cctype>

using namespace std;

enum NodeType { PROGRAM, DECL, ASSIGN, BINOP, ID, CONST };
enum DataType { INT_T, FLOAT_T, VOID_T, ERR_T };

struct ASTNode {
    NodeType nodeType;
    DataType dataType;
    string value;
    ASTNode* left;
    ASTNode* right;
};

ASTNode* makeNode(NodeType nt, DataType dt, string val, ASTNode*
l, ASTNode* r) {
    ASTNode* node = new ASTNode();
    node->nodeType = nt;
    node->dataType = dt;
    node->value = val;
    node->left = l;
    node->right = r;
    return node;
}

map<string, DataType> symbolTable;

DataType analyze(ASTNode* node) {
    if (!node) return VOID_T;

    if (node->nodeType == PROGRAM) {
        analyze(node->left);
        analyze(node->right);
        return VOID_T;
    }

    if (node->nodeType == DECL) {
        if (symbolTable.find(node->value) != symbolTable.end()) {
            cout << "Semantic Error: Redeclaration of variable '"
<< node->value << "'\n";
        } else {
            symbolTable[node->value] = node->dataType;
            cout << "Success: Declared '" << node->value << "'\n";
        }
        return VOID_T;
    }
```

```
    if (node->nodeType == ID) {
        if (symbolTable.find(node->value) == symbolTable.end()) {
            cout << "Semantic Error: Undeclared variable '" <<
node->value << "'\n";
            return ERR_T;
        }
        return symbolTable[node->value];
    }

    if (node->nodeType == CONST) {
        return node->dataType;
    }

    if (node->nodeType == BINOP) {
        DataType leftType = analyze(node->left);
        DataType rightType = analyze(node->right);

        if (leftType == ERR_T || rightType == ERR_T) return ERR_T;

        if (leftType != rightType) {
            cout << "Semantic Error: Type mismatch in '" << node-
>value << "' operation\n";
            return ERR_T;
        }
        return leftType;
    }

    if (node->nodeType == ASSIGN) {
        DataType leftType = analyze(node->left);
        DataType rightType = analyze(node->right);

        if (leftType == ERR_T || rightType == ERR_T) return ERR_T;

        if (leftType != rightType) {
            cout << "Semantic Error: Type mismatch in assignment
to '" << node->left->value << "'\n";
            return ERR_T;
        }
        cout << "Success: Assignment to '" << node->left->value <<
"' is semantically valid.\n";
        return leftType;
    }

    return VOID_T;
}

bool isNumber(const string& s) {
    return !s.empty() && (isdigit(s[0]) || s[0] == '.');
}

DataType getConstType(const string& s) {
    return (s.find('.') != string::npos) ? FLOAT_T : INT_T;
}

ASTNode* parseOperand(const string& s) {
    if (isNumber(s)) return makeNode(CONST, getConstType(s), s,
nullptr, nullptr);
    return makeNode(ID, VOID_T, s, nullptr, nullptr);
}
```

```
ASTNode* parseStatement(const vector<string>& tokens) {
    if (tokens.empty()) return nullptr;

    if (tokens[0] == "int") return makeNode(DECL, INT_T,
tokens[1], nullptr, nullptr);
    if (tokens[0] == "float") return makeNode(DECL, FLOAT_T,
tokens[1], nullptr, nullptr);

    if (tokens.size() >= 3 && tokens[1] == "=") {
        ASTNode* leftID = makeNode(ID, VOID_T, tokens[0], nullptr,
nullptr);
        ASTNode* rightNode = nullptr;

        if (tokens.size() == 3) {
            rightNode = parseOperand(tokens[2]);
        } else if (tokens.size() == 5) {
            ASTNode* op1 = parseOperand(tokens[2]);
            ASTNode* op2 = parseOperand(tokens[4]);
            rightNode = makeNode(BINOP, VOID_T, tokens[3], op1,
op2);
        }
        return makeNode(ASSIGN, VOID_T, "=", leftID, rightNode);
    }
    return nullptr;
}

int main() {
    cout << "Enter space-separated statements.\n";

    string line;
    ASTNode* root = nullptr;
    ASTNode* current = nullptr;

    while (true) {
        cout << "> ";
        getline(cin, line);
        if (line == "exit") break;

        stringstream ss(line);
        string token;
        vector<string> tokens;
        while (ss >> token) {
            tokens.push_back(token);
        }

        if (tokens.empty()) continue;

        ASTNode* stmt = parseStatement(tokens);
        if (stmt) {
            ASTNode* progNode = makeNode(PROGRAM, VOID_T, "",
stmt, nullptr);
            if (!root) {
                root = progNode;
                current = root;
            } else {
                current->right = progNode;
                current = progNode;
            }
        } else {
            cout << "Syntax Error: Unrecognized format.\n";
```

```
        }
    }

    cout << "\n--- Starting Semantic Analysis ---\n";
    analyze(root);

    return 0;
}
```

OUTPUT:

```
student@nitcse-OptiPlex-7000:~/24CS8017/LAB7$ g++ que_1.cpp -o q_1
student@nitcse-OptiPlex-7000:~/24CS8017/LAB7$ ./q_1
Enter space-separated statements.
> int x
> x = 10
> y = x + 5.5
> exit

--- Starting Semantic Analysis ---
Success: Declared 'x'
Success: Assignment to 'x' is semantically valid.
Semantic Error: Undeclared variable 'y'
Semantic Error: Type mismatch in '+' operation
student@nitcse-OptiPlex-7000:~/24CS8017/LAB7$
```